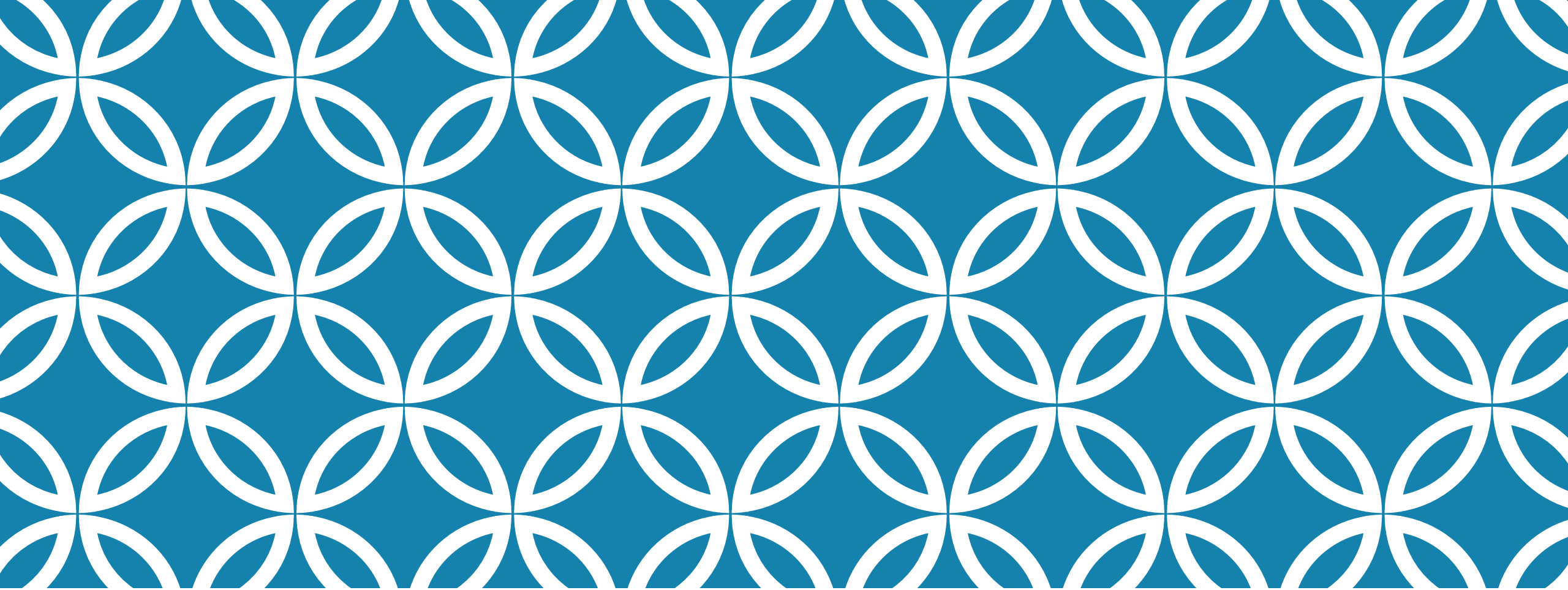


案例 2 · 动态待办清单（基础版）

让页面第一次「活」起来

ref / reactive → v-for + :key → v-model → @click → 修饰符

本案例学什么

- ref 定义基本类型，reactive 定义数组与对象
- ⚠ ref 在 <script> 里读写必须加 .value
- ⚠ reactive 数组不能整体赋值替换，要用 push / splice
- v-for 渲染列表，必须绑定 :key（严禁用 index）
- v-model 绑定输入框 —— 不用 getElementById 了
- @click 绑定添加 / 切换完成 / 删除三个方法
- uni.showToast 给出操作反馈（小程序里没有 alert）
- 修饰符：.trim / .number / @click.stop

产出：一个能增、能切换完成、能删、会统计的待办清单

案例运行结果


待办清单
请输入待办事项
添加
暂无待办，添加一条吧


待办清单
请输入待办事项
添加
复习数据结构
删除
开班会
删除
打球
删除
共 3 条 · 未完成 2 条
清空已完成

从「操作 DOM」到「操作数据」

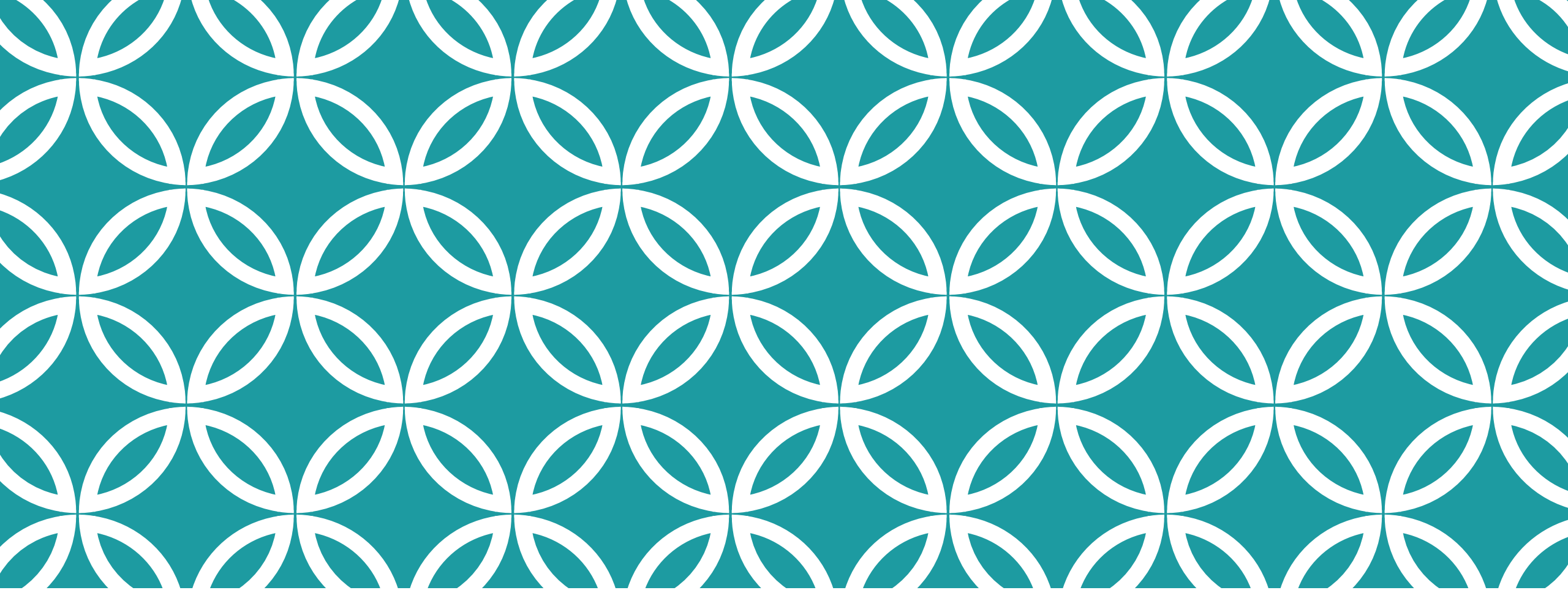
- 第一阶段的原生做法：找到节点 → 新建节点 → 挂上去 → 再改文字；How to do
- uni-app 里根本没有 DOM：小程序是 WXML，App 是原生 View
- 新做法：往数组里 push 一条，列表自己多出一行
- 数据变了视图自己变 —— 这就是「数据驱动」

第一阶段：原生 JS（改 DOM）

```
const ul =
document.getElementById('list')
const li = document.createElement('li')
li.textContent = newText
ul.appendChild(li)          // 手动改 DOM
```

本案例：Vue（改数据）

```
list.push({
  id: nextId++, text: newText,
done: false
})
// 就这一步，列表自己多出一行
```

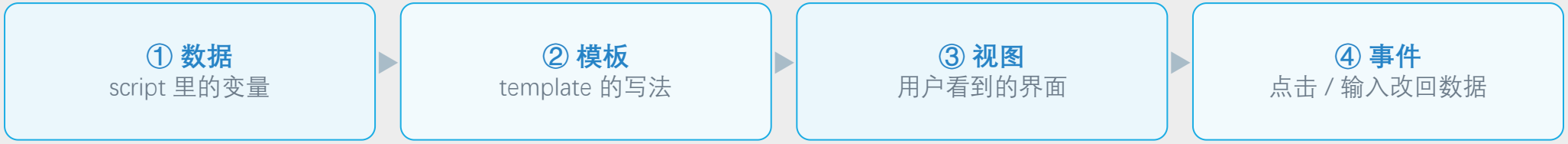


一、数据驱动与响应式

页面只是数据的投影，改数据就够了

什么是「数据驱动」

- 数据：script 里定义的变量，页面上所有内容的来源
- 模板：template 里的写法，声明「数据长什么样」
- 视图：用户看到的界面，由框架根据数据自动渲染出来



闭环：数据变 → 视图变；用户操作 → 改数据 → 视图再变

ref：基本类型的响应式

- ref 可以包任何类型：基本类型（字符串、数字、布尔）、对象、数组、Map、Set……
- script 里读写必须 .value —— 这是本案例第一个坑
- 模板里直接写变量名，框架自动解包，不用 .value
- 本案例的输入框内容 newText 就用 ref

```
const count = ref(0)         // 基本类型，只能 ref
const list = ref([])         // 数组，可以 ref
const user = ref({ name: 'a' })  // 对象，可以 ref

count.value++
list.value.push(1)
user.value.name = 'b'
```

```
import { ref } from 'vue'

const newText = ref('')         // 输入框里的内容

function addTodo() {
  console.log(newText.value)    // script 里读：必须 .value
  newText.value = ''            // 写：也要 .value
}

<!-- 模板里不用 .value -->
<input v-model="newText" />
```

ref 还是 reactive：怎么选

场景	用什么	
字符串 / 数字 / 布尔	ref	基本类型只能用 ref 包
对象 / 数组	reactive	直接改属性，不用写 .value
需要整体替换的数组	ref([])	script 里 list.value = 新数组
模板里取值	直接写名字	ref 自动解包，reactive 直接用
script 里改值	ref 要 .value	reactive 不要
解构展开	用 toRefs	reactive 直接解构会丢响应

reactive：数组与对象

- reactive 用来包对象或数组，本案例的待办列表就是它
- push / splice / 改属性，视图都会立刻更新
- reactive 不需要 .value，直接当普通对象用
- 注意：解构展开会丢掉响应式，需要时用 toRefs

```
import { reactive } from 'vue'

const list = reactive([])      // 待办数组

list.push({ id: 1, text: '买牛奶', done: false })  // 多一行
list[0].done = true                                 // 改属性
list.splice(0, 1)                                   // 删第 1 项
```

⚠ 坑一：ref 在 script 中必须 .value

- 写 newText = 某值：等于把变量整个换掉，响应式就断了
- 写 newText.value = 某值：改的是 ref 内部的值，视图才会更新
- 最难查的一类 bug：页面不动、控制台也不报错

```
// ✗ 错误：变量被重新赋值，响应式丢失
let newText = ref('')
newText = '买牛奶'              // 现在它只是个普通字符串

// ✓ 正确：改的是 ref 的 .value
newText.value = '买牛奶'        // 视图会跟着更新

// 模板里反过来：不用 .value
<text>{{ newText }}</text>
```

口诀：script 里看到 ref 变量，后面一定跟 .value

⚠ 坑二：reactive 数组不要整体赋值

- list = 新数组：变量指向了别处，模板盯着的还是旧数组
- 用 push / splice 原地修改，变量本身没换，响应就保住了
- 确实需要整体替换时，改用 ref([]) 再 list.value = 新数组

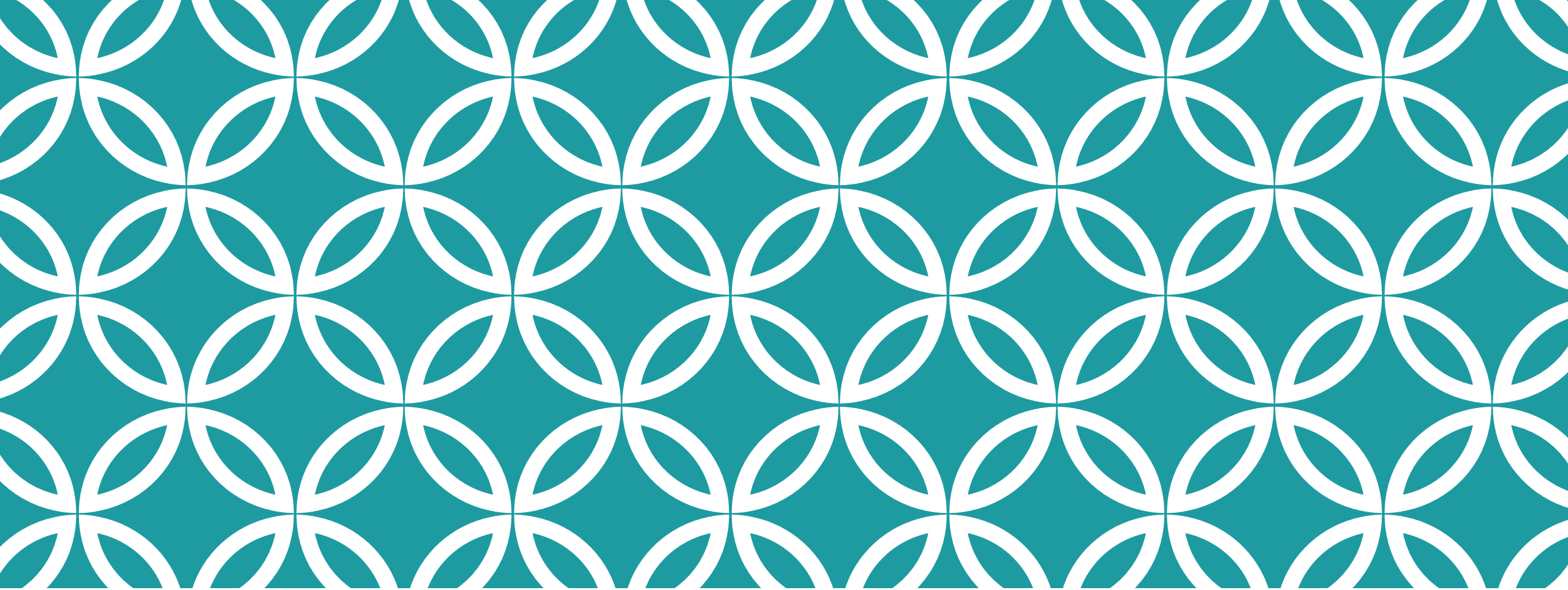
```
const list = reactive([])

// ✗ 错误：整体赋值，模板不再更新
list = list.filter(t => !t.done)

// ✓ 正确：原地修改
list.push({ id: 2, text: '写报告', done: false })
list.splice(0, 1)

// 需要整体替换时，用 ref 包数组
const list2 = ref([])
list2.value = list2.value.filter(t => !t.done)   // 合法
```

reactive 用 push / splice；ref 用 .value = 新数组



二、V-FOR：把数组渲染成列表

一个数组，N 行视图，还要认得出谁是谁

computed：未完成数量统计

- computed 是「算出来的数据」，依赖变了自动重算
- 第一阶段 DOM 讲义里讲过 computed，这里直接搬过来用
- 模板里当普通变量写，不用加括号
- 统计不需要手动更新 —— list 一变，数字自己变

```
import { computed } from 'vue'

// list 一变，unDoneCount 自动重算
const unDoneCount = computed(() => {
  return list.filter(t => !t.done).length
})
```

```
<text>共 {{ list.length }} 条 · 未完成 {{ unDoneCount }} 条</text>
```

v-for 基本语法

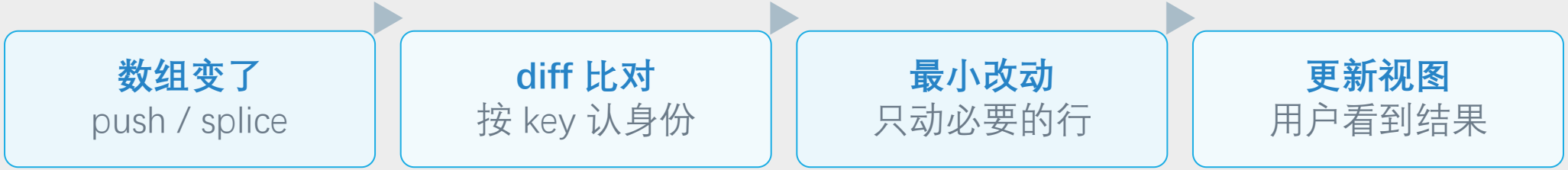
- v-for 写在要重复的那个标签上，uni-app 里通常是 view
- item 是当前这一项，可以在标签内部直接用
- uni-app 没有 ul / li，一行就是一个 view
- :key 必须写，值取每条数据自带的唯一 id

```
<view class="item" v-for="item in list" :key="item.id">
  <text>{{ item.text }}</text>
</view>

const list = reactive([
  { id: 1, text: '买牛奶', done: false },
  { id: 2, text: '写报告', done: true },
])
let nextId = 3          // 自增 id，每次 push 时 ++
```

为什么必须写 :key（diff 复用机制）

- 数组一变，框架要决定：哪些行能复用，哪些要新建或删掉
- 没有 key：按「位置」比对，第 1 行永远是第 1 行
- 有 key：按「身份」比对，认得出「还是原来那条数据」
- 有 key 时框架只做最小改动，性能更好、状态也不会错乱



key 就是每行数据的「身份证号」

⚠ 严禁用 index 当 key

- 删除第 1 项后，后面每一项的 index 都往前挪了一位
- 框架按位置复用 → 勾选状态整体错位
- 用自增 id：生成之后永不改变，才是真正的身份

```
<!-- ✗ 用 index：删除后位置全乱，勾选会错位 -->
<view v-for="(item, index) in list" :key="index">

<!-- ✓ 用数据自带的唯一 id -->
<view v-for="item in list" :key="item.id">

let nextId = 1
list.push({ id: nextId++, text, done: false })
```

id 在 push 时生成一次，之后永不改变 —— 这才是身份

v-for 里的 uni-app 标签

- 外层 view 包住一整行，文字必须放 text 里
- :class 绑定完成态，数据一变样式自动跟着变
- 对象写法 { done: item.done }：值为真才加这个类

```
<view class="item" v-for="item in list" :key="item.id">
  <text :class="{ done: item.done }">{{ item.text }}</text>
  <button size="mini" type="warn"
          @click.stop="deleteItem(item.id)">删除</button>
</view>

.done {
  text-decoration: line-through;
  color: #999999;
}
```

2. 对象语法 动态绑定class

当 `:class` 的值是一个**对象**时，规则是：

```
js

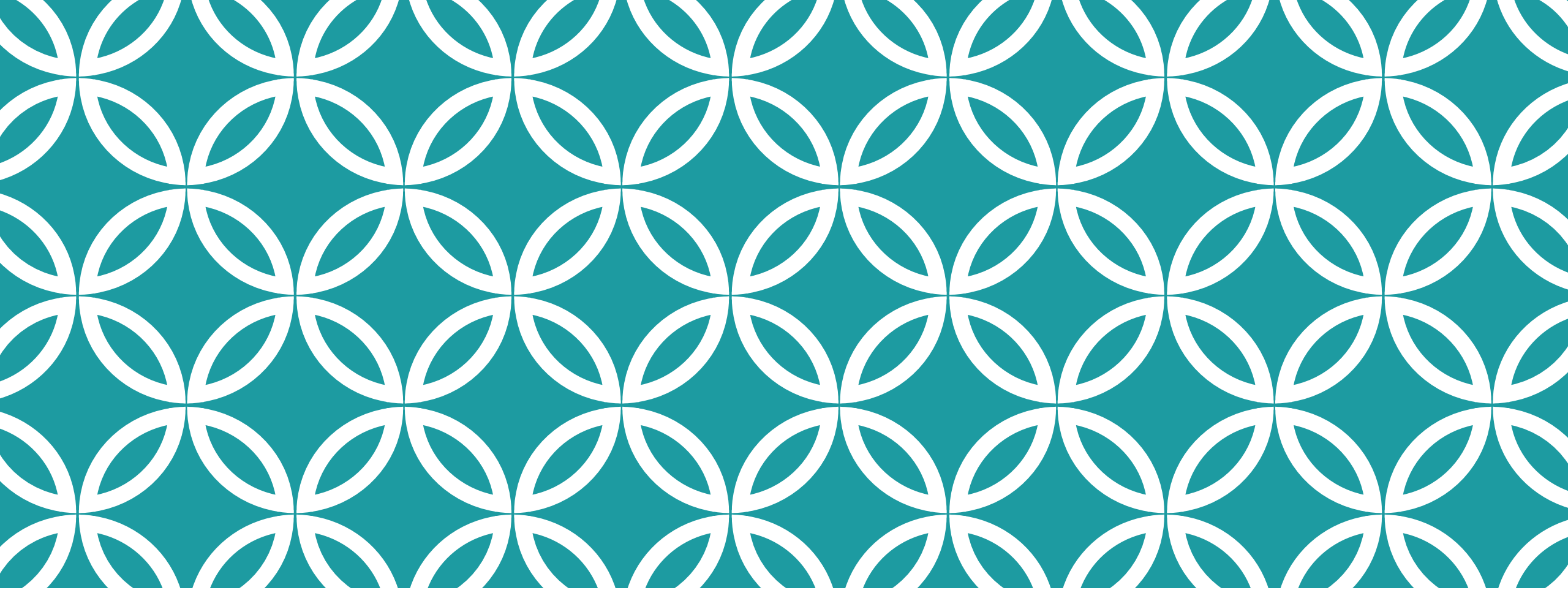
:class="{ 类名: 布尔值 }"
```

- 布尔值为 **true** → 加上这个类名
- 布尔值为 **false** → 不加这个类名

空列表的友好提示

- list 为空时给一句提示，比白屏友好得多
- v-if / v-else 是第一阶段学过的内容，写法完全一样
- v-if 和 v-for 不要写在同一个标签上（Vue3 里会报错）

```
<view v-if="list.length === 0" class="empty">
  <text>暂无待办，添加一条吧</text>
</view>
<view v-else class="list">
  <view class="item" v-for="item in list" :key="item.id">
    <text>{{ item.text }}</text>
  </view>
</view>
```



三、V-MODEL：双向绑定

不用 getElementById 了

v-model 到底做了什么

- v-model 一次干了两件事：把值绑过去 + 把输入收回来
- 原生写法要取节点、监听 input、手动赋值，这里一个属性搞定
- 输入框显示什么由数据决定，输入了什么数据立刻知道

第一阶段：原生 JS

```
const input =
document.getElementById('txt')
input.addEventListener('input', e =>
{
  newText = e.target.value   // 手动
同步
})
```

本案例：v-model

```
<input v-model="newText" />

// 一行顶上面四行
// 数据 ⇄ 输入框，双向同步
```

本案例的输入框与添加按钮

- input 标签在 uni-app 里同名，直接写
- v-model 绑到 newText，输入即同步到数据
- @click 绑到 addTodo 方法，点一次加一条
- placeholder 是输入框为空时的灰色提示

```
<view class="input-row">
  <input class="input" v-model.trim="newText"
         placeholder="请输入待办事项" />
  <button type="primary" size="mini"
          @click="addTodo">添加</button>
</view>
```

【补充】.trim 修饰符

- .trim 自动去掉输入内容的首尾空格
- 不加的话，输入三个空格也能成功添加一条空白待办
- 加了 .trim 之后，判空逻辑就可以直接写
- 修饰符可以叠加，如 v-model.trim.lazy

```
<!-- 不加修饰符：拿到什么就是什么 -->
<input v-model="newText" />
const text = newText.value.trim()      // 得自己 trim

<!-- 加了 .trim：拿到的已经是干净的 -->
<input v-model.trim="newText" />
const text = newText.value             // 直接用
```

【补充】.number 修饰符

- 输入框拿到的永远是字符串，哪怕 type 是 number
- 字符串做加法会变成拼接 —— 这是 JS 的经典坑
- 加 .number 后框架自动转成数字类型

```
<input v-model.number="age" type="number"
       placeholder="请输入年龄" />

// 不加 .number：拿到的是字符串
console.log('18' + 1)      // '181'    拼接
// 加了 .number：自动转成数字
console.log(18 + 1)        // 19
```

不加 .number，age + 1 得到的是 181 而不是 19

修饰符小结

修饰符	作用	用在哪	不加会怎样
.trim	去掉首尾空格	文本输入框	输入空格也能提交
.number	自动转成数字	数字输入框	'18' + 1 得到 '181'
.stop	阻止事件冒泡	嵌套的点击元素	父子点击一起触发
.lazy	失焦时才同步	长文本输入	边打字边更新数据

写法：v-model.修饰符="变量"　@事件名.修饰符="方法"　多个修饰符可叠加

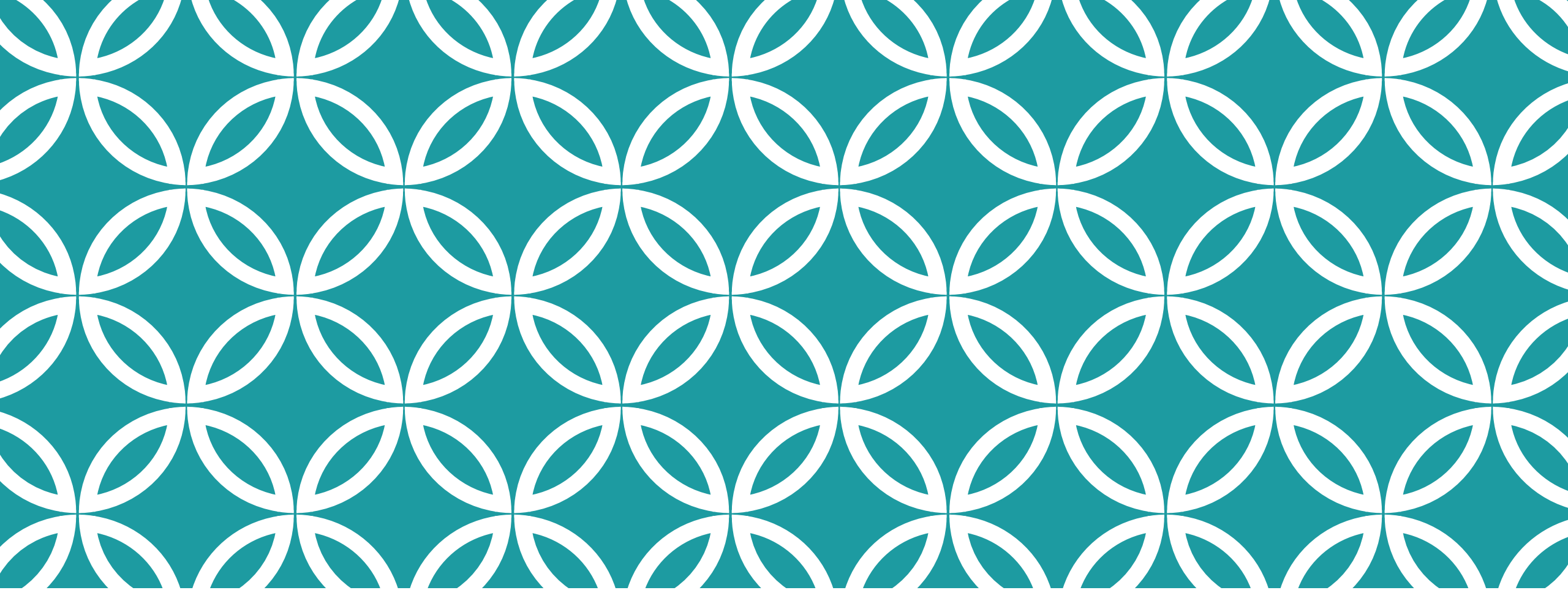
【扩展】回车 / 完成键添加：@confirm

- uni-app 的 input 有 confirm 事件：软键盘「完成」键触发
- 手机上没有回车键，靠的是键盘右下角那个按钮
- H5 里按回车同样会触发，两端写法统一
- confirm-type 专门用来设置手机软键盘右下角那个“确认键”的文字，它不触发事件。： done / send / search / next

```
<input v-model.trim="newText"
       placeholder="请输入待办事项"
       confirm-type="done"
       @confirm="addTodo" />

// 与点按钮走的是同一个方法，不用写两遍
```

复用 addTodo：@click 和 @confirm 指向同一个方法



四、事件绑定与方法

用户点了，数据就变，再给个反馈

@click 绑定添加方法

- addTodo 四步：取值 → 判空 → push → 清空并提示
- push 时带上 id: nextId++，为 :key 准备
- 取值和清空都要 .value（坑一），push 不整体赋值（坑二）

```
let nextId = 1
function addTodo() {
  const text = newText.value          // ref 必须 .value
  if (!text) {                        // 判空
    uni.showToast({ title: '请输入内容', icon: 'none' })
    return
  }
  list.push({ id: nextId++, text, done: false })
  newText.value = ''                  // 清空输入框
  uni.showToast({ title: '添加成功', icon: 'success' })
}
```

ES6 允许：当属性名和变量名相同时，可以只写一个。

uni.showToast：替代 alert

- 小程序和 App 里没有 alert，只有 uni 提供的 API
- 常用三个参数：title 文字、icon 图标、duration 时长
- icon 取 success 打勾 / none 纯文字 / loading 转圈
- 默认 1500 毫秒后自动消失，不用手动关闭

```
uni.showToast({ title: '添加成功', icon: 'success' })
uni.showToast({ title: '请输入内容', icon: 'none' })
uni.showToast({
  title: '加载中', icon: 'loading', duration: 2000
})
```

uni. 开头的是 uni-app 跨端 API，一套代码各端通用

点击整项切换完成状态

- @click="toggleDone(item)" 把当前这一项传进方法
- item.done = !item.done 取反，就这一行
- reactive 数组里的对象，改属性直接生效
- 删除线不用手动加：:class 跟数据绑定，数据变了样式自己变

```
<view class="item" v-for="item in list" :key="item.id"
      @click="toggleDone(item)">
  <text :class="{ done: item.done }">{{ item.text }}</text>
</view>

function toggleDone(item) {
  item.done = !item.done       // 改数据，删除线自己出现
}
```

【补充】事件修饰符 .stop

- 删除按钮在列表项里面，点它会一路「冒」到父级 view
- 不加 .stop：点一次删除，toggleDone 和 deleteItem 都执行
- 加上 @click.stop：冒泡被截断，只执行删除

```
<view v-for="item in list" :key="item.id"
      @click="toggleDone(item)">
  <text>{{ item.text }}</text>
  <!-- 加了 .stop 后，点删除不会触发父级的 toggleDone -->
  <button @click.stop="deleteItem(item.id)">删除</button>
</view>
```

嵌套点击必加 .stop，否则父子两个方法一起跑

冒泡是什么，.stop 干了什么

- 一次点击分三步：捕获（外→内）→ 目标 → 冒泡（内→外）
- 默认在冒泡阶段触发，所以子元素的点击会传到父元素
- .stop 相当于原生 e.stopPropagation()，一个修饰符搞定

① 捕获
从外向内找到目标

② 目标
真正点到的元素

③ 冒泡
从内向外往上传

```
// 原生写法：手动阻止冒泡
e.stopPropagation()

// Vue 写法：一个修饰符
<button @click.stop="deleteItem(item.id)">删除</button>
```

【扩展】清空已完成：坑二的实战

- 想清空已完成，第一反应都是 list = list.filter(...)
- 这正是坑二：reactive 数组整体赋值会丢响应
- 直接 list = remain 是让变量 list 指向了一个全新的普通数组，原来的响应式引用被切断，Vue 追踪不到变化，页面不更新
- 正确做法：splice 原地替换内容，变量本身不换

```
function clearDone() {
  // ✗ 整体赋值会丢响应（坑二）
  // list = list.filter(t => !t.done)

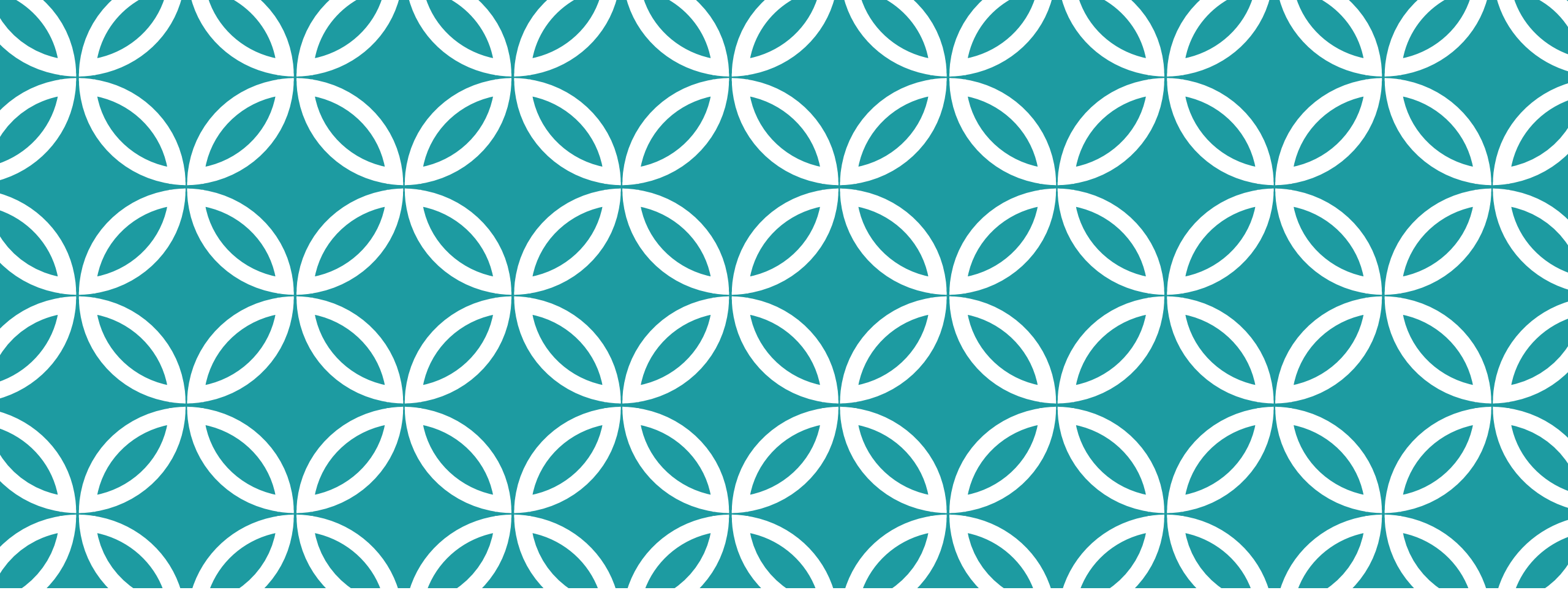
  // ✓ 原地改：删掉全部，再把剩下的插回去
  const remain = list.filter(t => !t.done)
  list.splice(0, list.length, ...remain)
  uni.showToast({ title: '已清空完成项', icon: 'success' })
```

splice(起点, 删几个, ...要插入的) —— 原地修改，保住响应式

```
arr.splice(start, deleteCount, ...items)
```

含义：

参数	作用
start	从哪个下标开始改
deleteCount	删除几个元素
...items	要插入的新元素（可选）



五、完整代码与实践任务

从 0 写出能增、能切、能删的待办清单

完整代码（上）：template

```
<template>
  <view class="page">
    <!-- 输入区：v-model 双向绑定 + @confirm 回车添加 -->
    <view class="input-row">
      <input class="input" v-model.trim="newText" placeholder="请输入待办事项"
             confirm-type="done" @confirm="addTodo" />
      <button type="primary" size="mini" @click="addTodo">添加</button>
    </view>
    <!-- 列表：v-for 必须绑 :key，值用 item.id，严禁用 index -->
    <view class="list">
      <view class="item" v-for="item in list" :key="item.id"
            @click="toggleDone(item)">
        <text :class="{ done: item.done }">{{ item.text }}</text>
        <button size="mini" type="warn" @click.stop="deleteItem(item.id)">删除</button>
      </view>
    </view>
    <view v-if="list.length === 0" class="empty">
      <text class="empty-text">暂无待办，添加一条吧</text>
    </view>
    <!-- 统计与清空 -->
    <view class="stat-row" v-if="list.length > 0">
      <text class="stat">共 {{ list.length }} 条 · 未完成 {{ unDoneCount }} 条</text>
      <text class="clear" @click="clearDone">清空已完成</text>
    </view>
  </view>
</template>
```

完整代码（中）：script setup —— 数据与添加

```
<script setup>
import { ref, reactive, computed } from 'vue'

const newText = ref('')       // 基本类型 → ref，要 .value
const list = reactive([])     // 数组 → reactive，不整体赋值
let nextId = 1                // 自增 id，严禁拿 index 当 :key

const unDoneCount = computed(() => {
  return list.filter(t => !t.done).length
})

function addTodo() {
  const text = newText.value            // ref 要 .value
  if (!text) {                          // 判空
    uni.showToast({ title: '请输入内容', icon: 'none' })
    return
  }
  list.push({ id: nextId++, text, done: false })
  newText.value = ''                    // 清空输入框
  uni.showToast({ title: '添加成功', icon: 'success' })
}
```

完整代码（中）：script setup —— 三个方法

```
// 点击整项：切换完成 / 未完成
function toggleDone(item) {
  item.done = !item.done      // 改数据，删除线自己出现
}

// 点删除按钮：.stop 保证不会触发父级的 toggleDone
function deleteItem(id) {
  const i = list.findIndex(t => t.id === id)
  if (i > -1) list.splice(i, 1)    // 原地改，保住响应
}

// 清空已完成：不能整体赋值，用 splice 原地替换
function clearDone() {
  const remain = list.filter(t => !t.done)
  list.splice(0, list.length, ...remain)
  uni.showToast({ title: '已清空完成项', icon: 'success' })
}
</script>
```

完整代码（下）：style scoped

```
<style scoped>
.page { padding: 30rpx; }
.input-row { display: flex; align-items: center; }
.input {
  flex: 1; height: 72rpx; padding: 0 20rpx; font-size: 28rpx;
  border: 1px solid #DDDDDD; border-radius: 8rpx;
}
.empty { padding: 120rpx 0; text-align: center; }
.empty-text { font-size: 28rpx; color: #999999; }
.item {
  display: flex; align-items: center;
  justify-content: space-between;
  padding: 28rpx 20rpx; margin-top: 20rpx;
  background: #FFFFFF; border-radius: 10rpx;
}
.item-text { font-size: 30rpx; color: #1A1A1A; }
.done { text-decoration: line-through; color: #999999; }
.stat-row {
  display: flex; justify-content: space-between;
  align-items: center; margin-top: 40rpx; padding: 0 20rpx;
}
.stat { font-size: 26rpx; color: #6B7A88; }
.clear { font-size: 26rpx; color: #2683C6; }
</style>
```

代码逐段拆解：三层各管什么



- 数据层：存什么 —— newText 存输入、list 存待办、unDoneCount 算统计
- 视图层：怎么显示 —— v-model 绑输入框、v-for 渲染每行、:class 管删除线
- 方法层：怎么改 —— 三个方法只改数据，一行 DOM 操作都没有
- 记住这条线：视图永远不会「被手动修改」，它只是数据的投影

任务要求与步骤拆解

- 1. 新建 uni-app 项目（Vue3 版本），pages.json 标题改成「待办清单」
- 2. 定义数据：newText 用 ref，list 用 reactive，nextId 自增计数器
- 3. 写输入区：input 绑 v-model.trim，button 绑 @click="addTodo"
- 4. 写列表：v-for + :key="item.id"，text 放文字，:class 绑完成态
- 5. 写三个方法：addTodo / toggleDone / deleteItem，都配 showToast
- 6. 加统计与清空：computed 算未完成数，clearDone 用 splice
- 7. 分别运行到浏览器和微信开发者工具，两端都要实际点一遍

交付前自查表

☐ 1. :key 绑的是 item.id 吗？（绝不能是 index）

☐ 2. <script> 里每个 ref 变量都写了 .value 吗？

☐ 3. 数组用的是 push / splice，没有整体赋值吗？

☐ 4. 删除按钮加了 .stop 吗？点删除会不会顺带切换状态？

☐ 5. 输入框加了 .trim 吗？只输入空格能挡住吗？

☐ 6. 每个操作都有 showToast 反馈吗？

☐ 7. 空列表时有友好提示吗？

☐ 8. 统计数字会跟着勾选自动变吗？

☐ 9. 清空已完成之后，剩下的项还在吗？数据还响应吗？

☐ 10. 浏览器和微信小程序两端都实际点过一遍吗？

常见翻车点与回顾

- 翻车 1：ref 忘写 .value → 页面毫无反应，还没报错
- 翻车 2：list = list.filter(...) → 数据变了视图不动
- 翻车 3：:key 用了 index → 删除后勾选状态整体错位
- 翻车 4：忘记 .stop → 点删除同时触发了切换
- 翻车 5：v-if 和 v-for 写在同一标签上 → 直接报错
- 一条线：数据（ref / reactive）→ 渲染（v-for + :key）→ 输入（v-model）→ 交互（@click + .stop）→ 反馈（showToast）
- 下个案例预告：把列表项抽成子组件 —— props 传进来，emit 传出去